VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

MICRO PROJECT REPORT

Project Title : Edge CDN Cache with Origin Shielding

SDG Aligned & Logo : SDG 9 – Industry, Innovation and Infrastructure

Project / Course Details	To be filled by students
Course / Subject Name	
Course Code / Semester / Academic Year	
Faculty Incharge / Project Mentor	
Project Group No.	
Submission Date	

Project Team Details

Student Name	Roll No.	Class / Division	Institute Email ID

Department of Computer Engineering

Academic Year: 2026–27

Table of Contents

1. Abstract
2. Introduction
3. Problem Definition, Objectives and Scope
4. Background / Literature Review
5. Proposed Solution and System Design
6. Methodology and Implementation
7. Testing, Evaluation and Results
8. Sustainability and SDG Alignment
9. Feasibility, Limitations and Risk Considerations
10. Conclusion and Future Scope
11. References
12. Appendices (if required)

Abstract

Popular websites such as online newspapers receive most of their traffic for a small set of objects, and that traffic arrives in bursts when news breaks. Serving every request from one origin server makes pages slow and lets a single failure take the whole site down. This micro project designs and builds a small tiered Content Delivery Network (CDN) called **Edge CDN** and drives it with a simulated newspaper, The Daily Edge. Requests pass through a router that balances load across three edge caches, each with its own Redis store. Edge misses go to a shared **Origin Shield**, which adds request collapsing, a bounded origin connection pool and a circuit breaker in front of the origin service and its MinIO object store. The system is written in Python with FastAPI and aiohttp, runs as independent Docker microservices, and can be deployed on Render. A live control-room dashboard shows hit ratios, latency and the serving layer of every request, and lets the user fail components on demand. In benchmarks on one development machine, warm edge caches cut average latency from 142.6 ms to 6.0 ms (23.8×), request collapsing reduced 200 simultaneous misses for one object to a single origin request, a Zipf-distributed workload reached an 81.8% cache hit ratio from cold caches, and every failure scenario (edge, shield or origin down) completed with a 100% success rate. By reducing origin load and keeping content available during outages, the project relates to SDG 9 (resilient infrastructure).

1. Introduction

1.1 Background and Motivation

A Content Delivery Network places copies of web content in caches close to users so that most requests never reach the origin server. News websites depend on CDNs heavily: a breaking story can multiply traffic within minutes, and readers expect the front page to load instantly. The project applies core system design concepts from the course: multi-level caching, cache expiry and eviction, load balancing with health checks, fault tolerance and graceful degradation, protection against overload (request collapsing, bounded pools and circuit breakers), microservice decomposition and observability through metrics.

The motivation is to make these ideas visible and measurable. Commercial CDNs are black boxes; by building a small one end to end, every request can be traced through each layer and every design decision can be benchmarked.

1.2 Problem Context

In a single-tier CDN each edge cache handles its own misses. When a new article is published, every edge misses at the same time and forwards its own request to the origin, so the origin receives one request per edge per object. During a traffic spike many readers also request the same uncached object at once (the “thundering herd”), and if the origin slows down or fails, readers see errors even though older copies of the content exist in the caches.

Origin shielding adds a shared second cache tier between the edges and the origin, so that one shield miss replaces many edge misses. Combined with request collapsing, circuit breaking and serving stale content on error, it lets a small origin survive large and bursty demand. The opportunity addressed here is to implement these mechanisms in a transparent system and quantify how much each one helps.

1.3 Intended Users / Stakeholders

- **Readers** of the newspaper, who want fast page loads and an available site.
- **Newsroom editors**, who publish breaking news and need it to appear on the homepage quickly.
- **Site operators / reliability engineers**, who monitor the CDN, tune TTLs and handle failures from the control-room dashboard.
- **Origin and infrastructure owners**, who benefit from lower origin load and cost.
- **Students and faculty**, who can use the system as a hands-on demonstration of CDN and caching concepts.

2. Problem Definition, Objectives and Scope

2.1 Problem Statement

Design and implement a tiered CDN for a high-traffic news website that serves most requests from caches near the reader, sends at most one request per object to the origin during a burst of identical misses, keeps serving content when an edge, the shield or the origin fails, and reports where each request was served and how long it took. The system must run on a single developer machine with Docker and on a free cloud hosting tier, using only open-source software.

2.2 Objectives

- **Design** a three-layer architecture (router, three edge caches, shared origin shield) in front of an object-storage origin.
- **Develop** each layer as an independent FastAPI microservice with its own Redis cache and Dockerfile.
- **Implement** TTL-based expiry per content class, LRU eviction, purge on publish and stale-if-error serving.
- **Implement** request collapsing, a bounded origin pool and a circuit breaker in the shield to protect the origin.
- **Automate** failure simulation and recovery for every component, with health checks and failover in the router.
- **Evaluate** hit ratio, latency (average, p95, p99), throughput, origin requests and availability with repeatable benchmark scenarios.
- **Build** a newspaper front end and a live control-room dashboard that show the serving layer of every request.

2.3 Scope and Deliverables

Included

- Router, three edge caches, origin shield, origin service, MinIO object store and a static web service, each in its own container.
- Synthetic newspaper content (articles, images, thumbnails, CSS/JS/SVG) generated and seeded automatically.
- Runtime configuration (TTLs, collapsing on/off, origin delay, timeouts), failure simulation, metrics API and benchmark suite.
- Unit and end-to-end tests, Docker Compose setup and Render deployment blueprint.

Excluded

- Geographic routing (GeoDNS/anycast), TLS termination, real user traffic, authentication of readers and paid CDN features such as image optimisation.

Deliverables

- Working prototype with source code, newspaper site, control-room dashboard, API documentation (OpenAPI at /docs), benchmark results and this report.

2.4 Assumptions and Constraints

- All edges run in one region; “nearest edge” is simulated by round-robin routing rather than by user location.
- Origin latency is simulated with a configurable delay (120 ms + 0–40 ms jitter in benchmarks) because origin and caches run on the same machine.
- Caches are in-memory Redis instances without persistence; edges are capped at 64 MB and the shield at 256 MB with allkeys-LRU eviction.
- Free hosting limits (512 MB RAM and 0.1 CPU per service on Render) restrict performance; benchmark figures were therefore measured locally.
- Content is fictional and generated by the project, so there are no copyright, privacy or external API constraints. No personal data is collected.

3. Background / Literature Review

The design draws on published work on CDN architecture and web caching, on the documentation of production CDNs and proxies, and on the HTTP caching standards. Table 1 summarises each source and how it shaped the project.

Table 1: Summary of related work

Source / Year	Approach or Key Finding	Strength / Limitation	Relevance to This Project
[1] E. Nygren, R. K. Sitaraman, J. Sun, 2010	Describes the Akamai platform: edge servers grouped in clusters, tiered distribution through parent caches to cut origin traffic.	Large-scale, real-world evidence; proprietary details are not public.	Basis for the edge → parent (shield) → origin hierarchy.
[2] L. Breslau et al., 1999	Web requests follow a Zipf-like popularity distribution; a small cache captures a large share of requests.	Strong empirical study; traffic mixes have changed since 1999.	Traffic generator and workload benchmark use Zipf popularity.
[3] Fastly, Shielding docs	A designated shield POP sits between edge POPs and the origin, so misses from all POPs are served by one location.	Clear, practical guidance; vendor-specific.	Shared Origin Shield service used by all three edges.
[4] Cloudflare, Tiered Cache docs	Lower-tier data centres ask upper-tier data centres before the origin, increasing hit ratio and reducing origin load.	Shows the benefit at scale; internal algorithms are not published.	Confirms the two-tier design and shield hit ratio as a metric.
[5] NGINX, proxy_cache_lock	Only one request at a time may populate a cache element; others wait for it.	Simple, proven technique; waiting requests add latency.	Request collapsing in the shield (one origin fetch per object).
[6] M. T. Nygard, Release It!, 2018	Circuit breaker and bulkhead patterns stop failures from cascading across services.	Widely adopted patterns; thresholds must be tuned per system.	Shield circuit breaker (5 failures, 10 s reset) and bounded origin pool.
[7] RFC 9111 (2022) and RFC 5861 (2010)	Define HTTP cache freshness, the Age header and the stale-if-error extension.	Standard and interoperable; the prototype implements the ideas, not full HTTP semantics.	TTL per content class, Age header and STALE-* responses when the origin is down.

3.1 Identified Gap and Project Contribution

Production CDNs implement shielding, tiered caching and collapsing, but they cannot be inspected or changed, and published research reports results at a scale a student team cannot reproduce. Simple teaching examples usually stop at a single cache. This project combines all of these mechanisms in one small, open system and adds observability that commercial services do not expose:

- every response states the layer that served it (X-Cache, X-Served-By, X-Latency-ms, X-Request-Id);
- each mechanism (collapsing, shield, edge cache, failover, stale serving) can be switched off or failed on demand and its effect measured;
- a realistic newspaper workload drives the CDN, so results reflect page-level traffic rather than synthetic single-key loops.

The contribution is therefore a practical integration and a measurement platform, not a new caching algorithm.

4. Proposed Solution and System Design

4.1 Proposed Solution

Edge CDN is a tiered cache that sits between readers and the newspaper's object store. Its key features are:

- **Router** – single public entry point; round-robin load balancing across healthy edges, health checks every second and per-request failover.
- **Three edge caches** – first-level caches, each with an independent Redis instance, TTL expiry and LRU eviction.
- **Origin Shield** – shared second-level cache that keeps objects twice as long as the edges, collapses concurrent misses, limits origin concurrency to four fetches and opens a circuit breaker after five consecutive origin failures.
- **Origin + MinIO** – serves articles, images, thumbnails and static files, adds configurable delay and handles publishing.
- **Graceful degradation** – edges bypass a failed shield, and expired copies are served as stale when the origin is unavailable.
- **Observability** – per-response headers, per-node counters, latency percentiles and a request log keyed by request ID.

User workflow. A reader opens the newspaper; the page and every image, thumbnail, stylesheet and script are fetched through the router. The CDN inspector on the page shows which layer served each object. An operator opens the control-room dashboard to watch hit ratios and latency, generate traffic, publish breaking news, change TTLs, fail components and run benchmarks.

4.2 System Architecture / Block Diagram / Conceptual Design

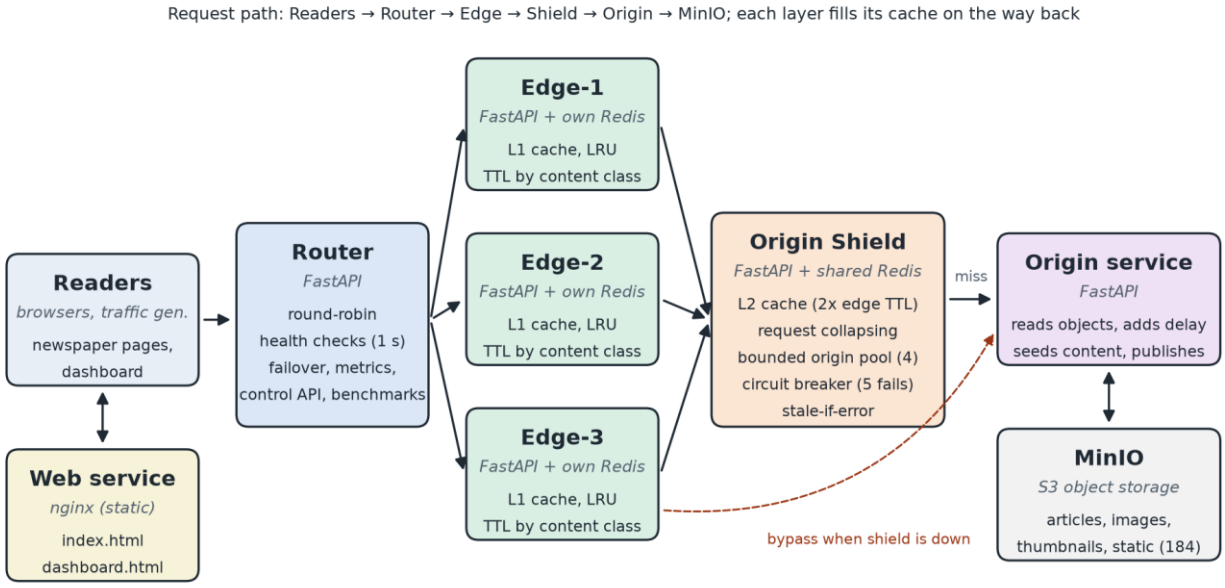


Figure 1: System architecture of Edge CDN with origin shielding

Figure 1 shows the components and the request path. Every response travels back along the same path, and each cache layer stores the object on the way back (cache-aside). Table 2 lists the three normal paths and the X-Cache value returned for each.

Table 2: Request paths through the CDN

Path	Route	X-Cache header
Edge hit	Reader → Router → Edge → Reader	HIT-EDGE
Shield hit	Reader → Router → Edge → Shield → Edge (fills) → Reader	HIT-SHIELD
Origin miss	Reader → Router → Edge → Shield → Origin/MinIO → Shield (fills) → Edge (fills) → Reader	MISS-ORIGIN
Collapsed miss	Same as origin miss, but waits for another request's fetch	MISS-COLLAPSED
Shield down	Edge fetches from the origin directly	MISS-ORIGIN-BYPASS
Origin down, copy expired	Expired copy returned by the edge or shield	STALE-EDGE / STALE-SHIELD

4.3 Module Description

Table 3: Module description

Module / Component	Responsibility	Input	Output
Router (services/router)	Public API, round-robin balancing, health checks, failover, metrics aggregation, control API, benchmarks	Reader HTTP requests; control commands	Response with X-Cache headers; metrics JSON
Edge ×3 (services/edge)	First-level cache with TTL and LRU; falls back to shield or origin	Object ID from router	Cached or fetched object; edge counters
Origin Shield (services/shield)	Shared second-level cache, request collapsing, bounded pool, circuit breaker, stale serving	Edge misses	Object for the edge; collapse and breaker statistics
Origin (services/origin)	Reads objects from MinIO, simulates delay, seeds content, publishes breaking news	Object ID; publish request	Object body with content class; new article
MinIO (services/minio)	S3-compatible object storage for all content	Object put/get	Stored articles, images and static files
Web (services/web)	Serves the newspaper and dashboard pages (nginx)	Browser request	index.html, dashboard.html
shared/cdncore	Common library: config, Redis cache store, metrics, admin API, HTTP helpers, request log	Used by all Python services	Consistent behaviour across nodes

4.4 Data / Inputs / Dataset (if applicable)

The project does not use an external dataset. Its inputs are synthetic newspaper content and generated traffic:

- **Content** – services/origin/content/generator.py creates 60 fictional articles in 8 sections (world, politics, business, technology, sports, science, health, culture), labelled breaking, popular, regular or archive. Each article has a hero image and a thumbnail rendered as JPEG with Pillow. Together with CSS, JavaScript and SVG files this gives 184 objects (about 4.4 MB) stored in MinIO.
- **Properties** – generation is deterministic, so every run has identical data; the content is invented, so there are no licensing, consent or privacy concerns.

- **Traffic** – the dashboard's traffic generator and the benchmark tool send requests with Zipf popularity, so a few hot articles receive most requests.
- **Test inputs** – benchmark scenarios (Table 9), failure toggles for each component and runtime configuration changes.

4.5 Algorithm / Design Decisions (if applicable)

Caching strategy and TTLs

Caches follow the cache-aside pattern. Each object has a content class that decides its time-to-live (Table 4). The shield keeps each object twice as long as the edges. Every Redis key lives for TTL + grace: after the TTL the copy is stale and is refetched on the next request, but it is still served if every upstream layer fails. Redis runs with maxmemory and the allkeys-lru policy, so memory pressure evicts the least recently used objects. Publishing breaking news purges index:latest from every cache.

Table 4: Time-to-live by content class

Content class	Edge TTL	Shield TTL
Breaking news	20 s	40 s
Homepage index	10 s	20 s
Popular articles	120 s	240 s
Regular articles	300 s	600 s
Archive articles	900 s	1,800 s
Images and thumbnails	1,800 s	3,600 s
Static files (CSS/JS/SVG)	3,600 s	7,200 s

Request handling at the shield (pseudocode)

```
on GET object_id:
    cached = redis.get(object_id)
    if cached and cached.fresh: return HIT
    try:
        if object_id in inflight:            # request collapsing
            return await inflight[object_id]  # MISS-COLLAPSED
        inflight[object_id] = new future
        breaker.before_call()                # raises if circuit open
        async with origin_pool(size=4):      # bounded concurrency
            obj = fetch(origin, object_id)
        breaker.record_success(); redis.set(object_id, obj, ttl*2 + grace)
        resolve future with obj; return MISS
    except origin error:
        breaker.record_failure()             # open after 5 in a row
        if cached (stale): return STALE-SHIELD
        raise 502
    finally: inflight.pop(object_id)
```

Circuit breaker

The breaker is closed while the origin works. After five consecutive failures it opens and rejects origin calls immediately for 10 s. It then moves to half-open and lets exactly one trial request through; success closes it and failure opens it again.

Key design decisions

- **Independent Redis per edge** so one cache failure or eviction storm does not affect the others; a separate Redis for the shield.
- **HTTP-only communication** between services so each can be deployed, scaled and failed independently.
- **aiohttp for internal hops** – with httpx the router reached about 390 req/s; with aiohttp about 1,860 req/s.
- **Bypass instead of failure** – when the shield is down, edges fetch from the origin directly, trading origin load for availability.

5. Methodology and Implementation

5.1 Workflow / Development Method

The project followed an iterative, incremental approach:

- **Requirements** – identified the required behaviours (tiered caching, shield, collapsing, TTLs, failure handling, metrics, benchmarks) from the project brief.
- **Design** – defined the layers, request paths, response headers and TTL classes; drew the architecture.
- **Prototype** – built the router, edges, shield and origin with the shared cdncore library and generated the newspaper content.
- **User interfaces** – added the newspaper with its CDN inspector and the control-room dashboard.
- **Microservices and deployment** – split the services into separate Docker images, added Docker Compose, a Render blueprint and a Render-like resource simulation.
- **Testing and evaluation** – wrote unit and end-to-end tests and a benchmark tool with six scenarios, then tuned the implementation from the results.
- **Operability** – added a /healthz liveness endpoint and a shared request ID logged by every service.

5.2 Technologies, Tools and Resources

Table 5: Technologies, tools and resources

Category	Technology / Tool	Purpose / Justification
Programming language / framework	Python 3.12; FastAPI 0.115–0.119; Uvicorn; aiohttp 3.10+	Asynchronous I/O suits a proxy that spends most time waiting on the network; FastAPI gives automatic OpenAPI docs.
Database / API / platform	Redis 7.4 (cache); MinIO (pgsty/minio build) object storage; Render	Redis provides TTL and LRU eviction natively; MinIO is S3-compatible and self-hosted; Render offers free container hosting.
Front end	HTML, CSS, JavaScript served by nginx 1.27	Lightweight newspaper and dashboard with no build step.
Development / testing tools	Docker and Docker Compose; pytest 8; custom benchmark tool (bench.py); Pillow 11; Git and GitHub	Reproducible multi-container setup, automated tests, repeatable measurements and image generation.
Hardware / cloud / other resources	Developer laptop; Render free plan (512 MB RAM, 0.1 CPU per service)	Local benchmarking and a public demo deployment at no cost.

5.3 Implementation Details

Router

Keeps the list of edges and a background task that checks each edge every second. Requests are sent to the next healthy edge in round-robin order; if the call fails the router retries on another edge (failover). It aggregates metrics from all nodes, exposes the control API (/failure, /config, /cache/flush, /api/publish) and runs the benchmark scenarios.

Edge caches

Each edge looks up the object in its Redis. A fresh copy is returned as HIT-EDGE. On a miss it asks the shield, stores the result with the edge TTL and returns it. If the shield is down it calls the origin directly; if both fail it serves an expired copy as STALE-EDGE.

Origin Shield

Implements collapsing with a dictionary of in-flight asyncio futures keyed by object ID, an asyncio semaphore of size four as the origin pool, and the circuit breaker described in Section 4.5. Collapsed waiters receive the same result or the same error.

Origin and content

Reads objects from MinIO, attaches the content class and content type, applies the configured delay and seeds or re-seeds the generated content on start.

Observability

Every response carries X-Cache, X-Served-By, X-Latency-ms and X-Request-Id. Each node counts hits, misses, collapses, stale responses and errors, and the router computes average, p50, p95 and p99 latency over the last 5,000 requests.

Challenges

- The upstream MinIO project stopped publishing images, so a maintained community build (pgsty/minio) is used.
- The first HTTP client (httpx) limited the router to about 390 req/s; switching internal hops to aiohttp raised it to about 1,860 req/s.
- Render's health checks would restart a service that was deliberately failed in a demo, so a separate /healthz endpoint stays healthy during simulated failures.
- Shell scripts broke in Linux containers when checked out with Windows line endings; LF endings are enforced through .gitattributes.

5.4 User Interface / Prototype / Output

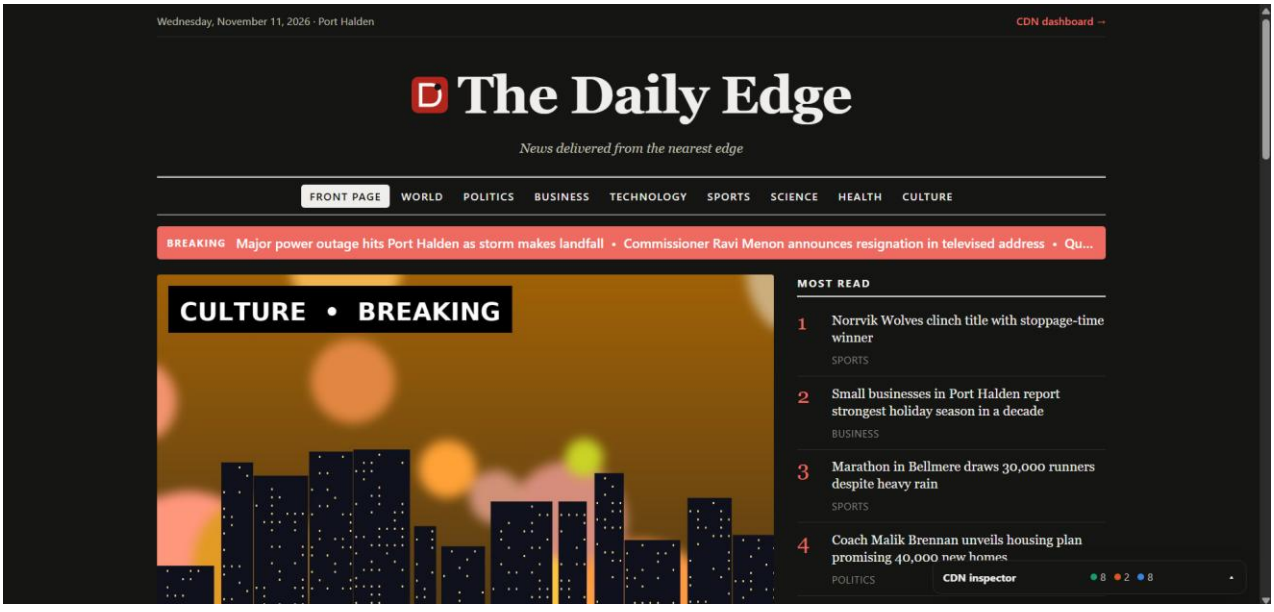


Figure 2: The Daily Edge newspaper served through the CDN

Figure 2 shows the reader-facing newspaper. All of its content – the breaking-news ticker, hero image, article list and most-read panel – is fetched through the router. The

CDN inspector in the bottom-right corner counts objects by serving layer (here 8 edge hits, 2 shield hits and 8 origin fetches), and the link at the top opens the dashboard.

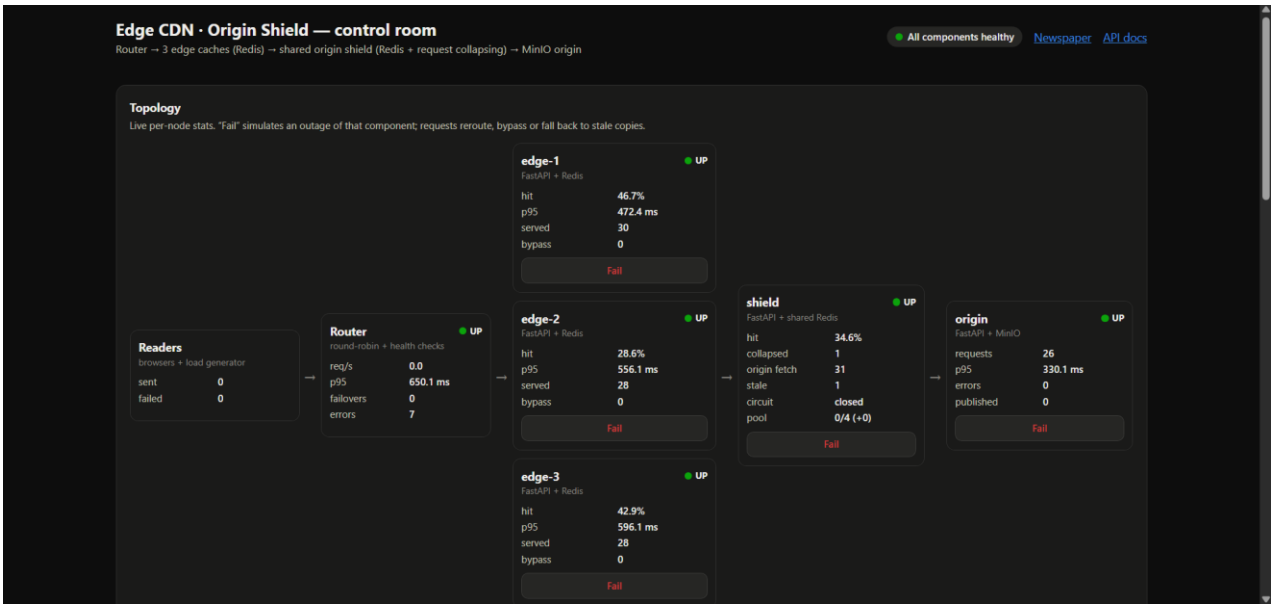


Figure 3: Control-room dashboard: live topology and per-node statistics

Figure 3 shows the live topology: readers, router, the three edges, the shield and the origin, each with its status, hit ratio, p95 latency and request counters. The shield panel also shows collapsed requests, origin fetches, stale responses, circuit state and pool usage. The Fail button on each node simulates an outage of that component.

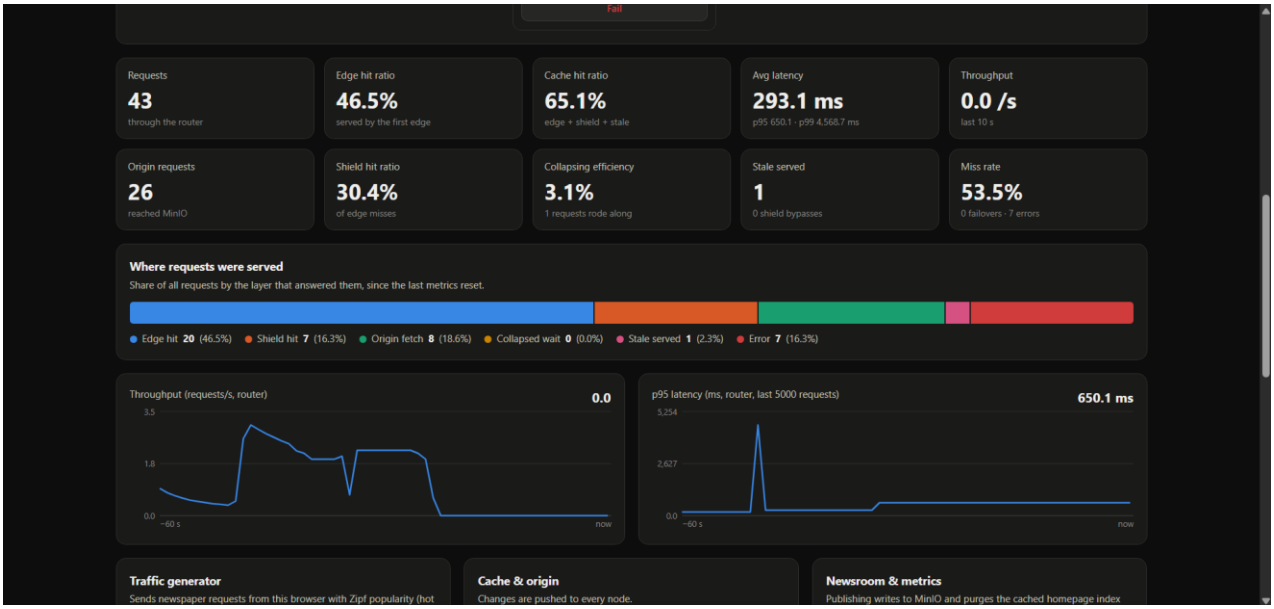


Figure 4: Dashboard summary metrics, serving-layer breakdown and live charts

Figure 4 shows the summary metrics (requests, edge and cache hit ratios, average latency with p95/p99, throughput, origin requests, shield hit ratio, collapsing efficiency, stale responses and miss rate), the breakdown of where requests were

served, and live throughput and p95 latency charts. The figure comes from a short manual session and illustrates the dashboard; the controlled measurements used for evaluation are reported in Section 6.

5.5 Team Contribution (for group projects)

Table 6: Team contribution

Team Member	Assigned Responsibility	Work Completed / Evidence
[Member 1 name]	Backend and caching layers	Router, edge and shield services, request collapsing, circuit breaker, TTL and eviction logic, unit tests
[Member 2 name]	Front end, deployment and evaluation	Newspaper and dashboard pages, content generator, Docker/Render deployment, benchmark runs, report

6. Testing, Evaluation and Results

6.1 Test Plan and Test Cases

Testing covered normal behaviour, concurrency limits and failure cases. Unit tests (pytest) check the cache logic in isolation; the benchmark tool exercises the full stack. Benchmarks were run with Docker Compose on one development machine without

CPU limits and with a 120 ms (+0–40 ms jitter) origin delay; raw results are in docs/benchmark-results.json. Table 7 lists the test cases.

Table 7: Test cases and results

Test ID	Test Input / Scenario	Expected Result	Observed Result	Status
TC-01	Same article requested cold, then via a warm shield, then via a warm edge	MISS-ORIGIN, then HIT-SHIELD, then HIT-EDGE	30/30 of each; avg 324 ms → 5.1 ms → 4.2 ms	Pass
TC-02	200 simultaneous requests for one uncached image, collapsing on	One origin request; others wait	1 origin request, 199 MISS-COLLAPSED, 100% success	Pass
TC-03	Same as TC-02 with collapsing off (baseline)	Many duplicate origin requests	172 origin requests, p95 5,853 ms	Pass
TC-04	Edge-1 failed during 300 requests	Traffic moves to edge-2 and edge-3; no errors	300/300 succeeded	Pass
TC-05	Shield failed during 300 requests	Edges bypass to origin; no errors	300/300 succeeded; 156 MISS-ORIGIN-BYPASS	Pass
TC-06	Origin failed, caches fresh	Served from caches	300/300 HIT-EDGE	Pass
TC-07	Origin failed, cached copies expired	Stale copies served instead of errors	300/300 succeeded; 137 STALE-SHIELD	Pass
TC-08	Five consecutive origin failures, then recovery	Breaker opens, rejects calls, half-opens after reset, closes on success	Unit test passed	Pass
TC-09	Origin error while several requests are collapsed	Single fetch; every waiter gets the error	Unit test passed	Pass
TC-10	TTL per class, shield factor and runtime config parsing (booleans, TTLs)	Correct TTLs; invalid values handled	Unit tests passed	Pass

6.2 Evaluation Metrics

Metrics were chosen to show cache effectiveness, speed, origin protection and availability. Values below are from the Zipf workload benchmark (1,500 requests, cold caches) unless stated otherwise.

Table 8: Evaluation metrics

Metric	Measurement Method / Formula	Observed Value	Interpretation
Edge hit ratio	HIT-EDGE responses ÷ total requests	75.1%	Three-quarters of requests never left the first cache layer.
Cache hit ratio	(edge hits + shield hits + stale) ÷ total	81.8%	Only 18.2% of requests needed the origin, even starting from empty caches.
Origin requests	Count of fetches reaching MinIO	162 of 1,500	About 108 origin requests per 1,000 reader requests.
Collapsing efficiency	collapsed ÷ (collapsed + origin fetches)	40.7% (111 collapsed)	Concurrent misses for the same object shared one fetch.
Latency (avg / p50 / p95)	Client-side timing per request	146 / 1.3 / 1,114 ms	Median is very low; the tail comes from cold-start origin fetches queued behind the 4-slot pool.
Throughput	Completed requests ÷ wall time	252 req/s (workload); 3,050 req/s (warm edge)	Warm caches serve over 20× more requests than the origin path.
Availability under failure	Successful responses ÷ requests in each failure phase	100% in all 4 phases	No reader-visible errors while edge, shield or origin was down.

6.3 Results and Discussion

Table 9: Benchmark results

Comparison	Baseline	With CDN mechanism	Improvement
Direct origin vs warm edge (same hot articles, 300 requests)	142.6 ms avg; 135 req/s	6.0 ms avg; 3,050 req/s	23.8× faster
Origin fetch vs shield hit vs edge hit (30 requests each)	324 ms avg (origin)	5.1 ms (shield); 4.2 ms (edge)	~63× / ~78× faster
200 simultaneous misses for one image	172 origin requests; p95 5,853 ms	1 origin request; p95 196 ms	99.4% fewer origin requests
Origin delay 0 ms vs 300 ms	Cold: 10 ms vs 643 ms	Warm: ~3.3 ms in both	Cache hides origin slowness
Zipf workload, cold caches (1,500 requests)	–	81.8% cache hit ratio; 162 origin requests	–
Failure phases (300 requests each)	–	Edge-1 down, shield down, origin down, origin down + expired: 100% success	No errors

All objectives were met. The edge cache gives the largest latency gain, while the shield turns an edge miss into a ~5 ms response instead of a ~324 ms origin fetch. Request collapsing is the most important protection for the origin: without it, 200 concurrent readers of a new image produced 172 origin requests and a p95 of almost 6 s; with it, one request and a p95 under 200 ms.

Failure handling worked in every phase. With edge-1 down the edge hit ratio actually rose (67% → 90%) because the remaining two edges received more repeat traffic. With the shield down latency rose to 75 ms on average because 156 requests bypassed to the origin, which is the expected cost of losing the shield. With the origin down and copies expired, 137 stale responses kept the site fully available.

Unexpected or limiting results: the cold workload has a high p95 (1.1 s) because many different objects miss at once and queue for the four-slot origin pool; a larger pool would lower the tail at the cost of more origin load. All figures were measured on one machine; on Render's free plan (0.1 CPU per service) the same scenarios are much slower.

6.4 Validation / Demonstration Evidence

The features can be demonstrated live as follows:

- Start the stack with docker compose up --build and open the newspaper (localhost:8080) and dashboard (localhost:8080/dashboard.html).
- Reload the newspaper and watch the CDN inspector move from origin fetches to edge hits.
- Start the traffic generator and watch hit ratio, latency and the serving-layer bar update.
- Click Fail on an edge, the shield and the origin in turn; the newspaper keeps loading and the dashboard shows failovers, bypasses and stale responses.
- Publish breaking news; the article appears on the homepage after the index purge.
- Run the benchmark from the dashboard and compare with Table 9.

Source code: github.com/ShreyashSingh857/SSD-MiniProject. The unit test suite was run again while preparing this report: 9 passed; the 6 end-to-end tests run only against a live stack (CDN_URL set). No passwords or tokens are included in this report.

7. Sustainability and SDG Alignment (Mandatory for Every Project)

This section assesses the environmental, social and economic effects of the project. The main link is efficient and resilient digital infrastructure; other links are indirect and are stated as such.

7.1 Sustainability Assessment

Table 10: Sustainability assessment

Dimension	Questions to Consider	Project-specific assessment / evidence
Environmental	Energy and compute use; paper/material use; e-waste; resource efficiency; emissions; lifecycle impacts.	Positive (measured indirectly): caching served 81.8% of workload requests without the origin, and collapsing cut 172 origin fetches to 1, reducing origin compute and data transfer. Negative: extra Redis containers use memory (3 × 64 MB + 256 MB) and idle services consume energy. No paper or hardware is used beyond an existing laptop.
Social	Accessibility and inclusion; user safety; privacy; fairness; digital divide; community benefit.	Positive: faster pages help readers on slow connections, and stale serving keeps news available during outages, when timely information matters most. No personal data is collected or stored. Accessibility of the pages was not formally evaluated.
Economic	Development and operating cost; maintenance; affordability; productivity; long-term viability.	Positive: development cost Rs. 0 (open-source stack, free hosting). Lower origin load means a smaller, cheaper origin can serve the same audience. Trade-off: seven services are more work to operate and maintain than a single server.

7.2 UN Sustainable Development Goals (SDGs)

Only goals with a clear link to a project feature are listed. SDG 9 is the direct link; the others are indirect and their contribution is a reasoned assessment rather than a measurement.

Table 11: SDG alignment

SDG No. and Name	Project Feature / Activity Linked to SDG	Expected or Observed Contribution	Evidence / Indicator
SDG 9: Industry, Innovation and Infrastructure (target 9.1, resilient infrastructure)	Tiered caching, failover, shield bypass, stale-if-error	Direct: keeps a web service available when components fail and reduces load on the origin	100% success in all four failure phases; 81.8% cache hit ratio
SDG 16: Peace, Justice and Strong Institutions (target 16.10, public access to information)	Newspaper stays readable during origin outages and traffic spikes	Indirect: more reliable access to news during events such as storms or power cuts	137 stale responses served instead of errors with the origin down
SDG 7: Affordable and Clean Energy (target 7.3, energy efficiency)	Request collapsing and caching avoid repeated origin work	Indirect and not measured: fewer origin fetches should mean less compute per reader	Origin requests 172 → 1 (collapsing); energy not measured

7.3 Sustainability Measures and Trade-offs

- **Efficient algorithms** – request collapsing and the bounded origin pool avoid duplicate work during bursts.
- **Resource caps** – Redis memory limits with LRU eviction keep each cache small; TTLs per content class avoid storing content longer than needed.
- **Responsible cloud usage** – the free plan scales idle services down; benchmarks run locally instead of loading cloud resources.
- **Data minimisation** – only synthetic content is stored; no reader data is logged beyond anonymous request IDs.

Trade-offs. Longer TTLs raise the hit ratio and lower origin load but can show outdated news, which is why breaking news has a 20 s TTL and publishing purges the index. Serving stale copies keeps the site up but may show old content. More cache memory raises the hit ratio but uses more resources.

7.4 Sustainability Indicator / Follow-up

- **Origin requests per 1,000 reader requests** – measured: about 108 in the cold Zipf workload (162 of 1,500).

- **Availability during component failure** – measured: 100% in each failure phase.
- **Bytes served from cache vs origin** – proposed future measure of network savings.
- **CPU-seconds or energy per 1,000 requests, with and without caching** – proposed future measure (e.g. container CPU accounting).

8. Feasibility, Limitations and Risk Considerations

8.1 Feasibility

- **Technical** – feasible with well-documented open-source tools; the prototype is complete and tested.
- **Time and resources** – built within the micro-project timeline by two members on personal laptops.
- **Operational** – starts with one command (docker compose up); the dashboard exposes all controls without using the command line.
- **Cost** – no spending was required (Table 12).

Table 12: Estimated cost

Item / Resource	Estimated Cost (Rs.)	Basis / Notes
Hardware / hosting / cloud	0	Existing laptops; Render free plan for the demo
Software / API / dataset	0	Python, FastAPI, Redis, MinIO, nginx, Docker – all free and open source; content generated by the project
Other resources	0	None
Total estimated cost	0	One-time: Rs. 0. Recurring: Rs. 0 on the free plan; a paid hosting plan would add a monthly cost for production use.

8.2 Limitations

- All edges run in one location, so the geographic latency benefit of a real CDN is simulated, not measured.
- Benchmarks come from one machine without CPU limits; the free hosting tier is much slower and services sleep when idle.
- The router is a single instance and therefore a single point of failure.
- Caches are not persistent; a restart empties them and the hit ratio starts from zero.

- Only the homepage index is purged on publish; there is no tag-based purge or HTTP revalidation (ETag / 304).
- Content and traffic are synthetic; real readers may show different popularity patterns.

8.3 Risks, Ethics, Privacy and Security

- **Access control** – the control endpoints (fail, flush, config, publish) can disrupt the service. When CONTROL_TOKEN is set every POST requires an X-Control-Token header; it must be set on any public deployment.
- **Credentials** – MinIO's development credentials must be replaced in production; tokens are kept in environment variables, not in code.
- **Stale or incorrect content** – a corrected article may stay cached until its TTL expires; short TTLs for breaking news and purge-on-publish reduce this.
- **Cache poisoning / overload** – the CDN only caches objects returned by the origin, and the bounded pool and circuit breaker stop overload from cascading.
- **Privacy and ethics** – no personal data is collected. All news content, places and people are fictional, so the site cannot spread real misinformation.
- **Transport security** – TLS is provided by the hosting platform on Render; the local setup uses plain HTTP.

9. Conclusion and Future Scope

9.1 Summary

The project delivered a working tiered CDN with origin shielding: a router with health checks and failover, three independent edge caches, a shared shield with request collapsing, a bounded origin pool and a circuit breaker, and an origin backed by MinIO, all as separate microservices with a newspaper front end and a live dashboard. Every objective was met and measured: warm edges were 23.8× faster than the origin, collapsing reduced 200 concurrent misses to one origin request, a cold Zipf workload reached an 81.8% cache hit ratio, and the site stayed 100% available with an edge, the shield or the origin down.

From a sustainability perspective the project shows how caching and overload protection make web infrastructure more resilient and reduce repeated origin work (SDG 9), with indirect benefits for access to information during outages.

9.2 Future Scope

- Geographic routing (latency-based or GeoDNS) and edges deployed in several regions.
- Consistent hashing across edges to reduce duplicated objects.
- HTTP revalidation (ETag, If-None-Match), stale-while-revalidate and tag-based purge.
- A replicated router behind a load balancer to remove the single point of failure.
- Prometheus and Grafana for long-term metrics; alerts on hit ratio and error rate.
- Comparison of eviction and admission policies (LRU, LFU, TinyLFU).
- Measuring CPU time and energy per 1,000 requests with and without the CDN.

References

[1] E. Nygren, R. K. Sitaraman, and J. Sun, “The Akamai network: A platform for high-performance Internet applications,” ACM SIGOPS Operating Systems Review, vol. 44, no. 3, pp. 2–19, 2010.

[2] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, “Web caching and Zipf-like distributions: Evidence and implications,” in Proc. IEEE INFOCOM, 1999, pp. 126–134.

[3] Fastly, “Shielding,” Fastly Documentation. [Online]. Available: https://docs.fastly.com/en/guides/shielding (accessed Nov. 2026).

[4] Cloudflare, “Tiered Cache,” Cloudflare Docs. [Online]. Available: https://developers.cloudflare.com/cache/how-to/tiered-cache/ (accessed Nov. 2026).

[5] NGINX, “Module ngx_http_proxy_module: proxy_cache_lock,” nginx.org documentation. [Online]. Available: https://nginx.org/en/docs/http/ngx_http_proxy_module.html (accessed Nov. 2026).

[6] M. T. Nygard, Release It! Design and Deploy Production-Ready Software, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[7] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Caching,” RFC 9111, IETF, Jun. 2022.

[8] M. Nottingham, “HTTP Cache-Control Extensions for Stale Content,” RFC 5861, IETF, May 2010.

[9] Redis Ltd., “Key eviction,” Redis Documentation. [Online]. Available: https://redis.io/docs/latest/develop/reference/eviction/ (accessed Nov. 2026).

[10] FastAPI, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/ (accessed Nov. 2026).

[11] MinIO, Inc., “MinIO object storage documentation.” [Online]. Available: https://min.io/docs/ (accessed Nov. 2026).

[12] Render, “Render documentation.” [Online]. Available: https://render.com/docs (accessed Nov. 2026).

Appendices

Appendix A: Router API endpoints

Table 13: Router API

Method	Endpoint	Description
GET	/article/{article_id}	Request an article through the CDN
GET	/content/{object_id}	Request any cacheable object, e.g. image:world-013, static:homepage.css, index:latest
GET	/direct/{object_id}	Bypass the CDN (benchmark baseline)
GET	/cache/status	Keys, memory and evictions per cache; circuit and pool state
POST	/cache/flush	Flush all caches, edges, shield or one edge (optionally one object)
GET / POST	/metrics, /metrics/reset	Hit ratios, latency percentiles, throughput, origin requests, collapsing efficiency, per-node counters; reset
GET	/health, /healthz	Component availability; liveness for the hosting platform
POST	/failure/{component}	Fail or restore edge-1..3, shield or origin; /failure/reset restores all
GET / POST	/config	Runtime TTLs, collapsing, origin delay and timeouts, pushed to every node
POST	/api/publish	Publish a breaking-news article and purge the homepage index
GET / POST	/api/benchmark	Start a benchmark scenario or read its results

Appendix B: Request collapsing in the shield (excerpt, services/shield/shield_app.py)

```
async def collapsed_fetch(object_id):
    if not state.config.collapsing:
        return await fetch_from_origin(object_id), False
    pending = inflight.get(object_id)
    if pending is not None:
        state.metrics.inc("collapsed")
        return await asyncio.shield(pending), True
    future = asyncio.get_running_loop().create_future()
    inflight[object_id] = future
    try:
        result = await fetch_from_origin(object_id)
        future.set_result(result)
        return result, False
    except Exception as exc:
        future.set_exception(exc)
        raise
    finally:
        inflight.pop(object_id, None)
```

Appendix C: Automated test suite

Table 14: Unit and end-to-end tests

Test	Type	What it checks	Latest result
test_parse_edges	Unit	Edge list parsing from configuration	Passed
test_ttl_per_class_and_shield_factor	Unit	TTL per content class; shield 2×	Passed
test_config_update_parses_booleans_and_ttls	Unit	Runtime config validation	Passed
test_metrics_percentiles	Unit	p50/p95/p99 calculation	Passed
test_generator_is_deterministic_and_complete	Unit	Same 184 objects every run	Passed
test_circuit_breaker_opens_and_recovers	Unit	Closed → open → half-open → closed	Passed
test_request_collapsing_single_origin_fetch	Unit	Concurrent misses → one fetch	Passed
test_without_collapsing_every_request_hits_origin	Unit	Baseline without collapsing	Passed
test_collapsed_waiters_share_failure	Unit	Errors reach every waiter	Passed
test_tiered_cache_path, test_edge_failover, test_shield_bypass_when_shield_down, test_cached_content_survives_origin_failure, test_publish_purges_index, test_metrics_and_health_shape (6 tests)	End-to-end	Full request paths and failures against a running stack	Run with CDN_URL set (skipped without a live stack)

Appendix D: Repository and deployment

- Repository: github.com/ShreyashSingh857/SSD-MiniProject
- Local run: docker compose up --build – newspaper on port 8080, router API docs at localhost:8000/docs.
- Render-like limits locally: docker compose -f docker-compose.yml -f docker-compose.render-sim.yml up --build
- Cloud: render.yaml deploys each service separately on Render; settings are listed in docs/RENDER_DEPLOYMENT.md.
- Benchmarks: services/router/bench.py --scenario all; results in docs/benchmark-results.json.